PROJECT TECHNICAL REPORT: PERSISTENT MULTIPLATFORM MARKETPLACE WITH AUTOMATED CHAT ARCHITECTURE

Authors: Irene Ferrández Colomer, María García Vega, Alba López Martínez, Alba Lozano Guixa

Framework: Flutter (Dart)

Target Platform: https://github.com/alopmar5/flutter_application_1

1. Executive Summary & Project Intent

1.1 Problem Statement and Real-World Motivation

The conception of this project stems directly from a practical, widespread issue faced by international student communities, specifically Erasmus students living in university residences. At the end of each academic cycle or semester, thousands of exchange students face the critical challenge of managing heavy, bulky, or semi-permanent assets—such as bedding, kitchenware, desk electronics, study materials, and small appliances. Due to strict airline baggage restrictions, high international shipping costs, or logistical limitations, these students cannot transport these perfectly functional goods back to their home countries.

Consequently, a massive volume of high-value items is either prematurely discarded or left abandoned in residence common rooms. Concurrently, new incoming Erasmus students arrive every semester facing the immediate financial burden of purchasing these exact same items brand new. This cyclical mismatch highlights a clear inefficiency in local student housing communities.

1.2 Proposed Solution

To resolve this issue, this project introduces a localized, peer-to-peer digital marketplace tailored specifically for student residences. The application provides a frictionless platform where departing students can instantly list their items for sale or donation, and arriving students can easily discover and acquire them before setting foot in the country. By keeping transactions entirely within the residence perimeter, the platform eliminates delivery logistics, supports the financial well-being of international students, and promotes a highly sustainable circular economy within university housing.

The application architecture supports user registration, credential authentication, a dynamic and searchable product catalog, automated transactional histories (including separate user modules for bookmarked items and finalized purchases), live identity modification, and a smart, asynchronous chat module operating in English to simulate natural human-agent negotiations.

2. Structural Architecture & Component Design

The system implements a Layered Architecture combined with a Centralized Single Source of Truth (SSOT) data pattern. By explicitly decoupling data presentation layers from state mutations and memory allocations, the application enforces high cohesion and loose coupling. This design reduces regressions during feature iterations and software updates.

2.1 Component Interaction Pipeline

The system's architectural topology is divided into three distinct operational domains: the Routing Layer, the State Representation Layer, and the In-Memory Data Tier.

- **The Routing Gateway:** Governs user progression and session lifecycle. It relies on a declarative routing table mapping explicit system paths. This mapping ensures structural isolation, preventing overlapping layouts and allowing the view stack to clear safely during authentication changes.
- **The View and Presentation Layer:** Manages user interaction and interface construction. Components within this tier handle operations like searching, form state caching, profiling, and interaction routing. These components respond to input changes by triggering local state updates.
- **The Central Data Store:** Functions as an abstract relational broker, acting as a local memory cache for application variables. It standardizes mutations across all modules, tracking the active session context, published marketplace items, user inventory, favorite products, and completed purchase transactions.

2.2 Functional Module Breakdown

The system achieves modular decoupling through independent functional screens, each addressing a specialized aspect of the application lifecycle:

- **Authentication & Guard Layer:** Coordinates validation rules against the relational database mock. It checks input integrity and securely assigns user entities to the session registry upon successful validation, shifting the app context to the authenticated workspace.
- **Feed & Asset Explorer:** Manages search indexing and dynamic view rendering. It utilizes an optimized virtualized collection manager that allocates visual components dynamically based on viewport constraints, preventing performance lags during heavy catalog browsing.
- **Form & Data Marshalling Module:** Tracks multi-field user input states. It preserves typed inputs through specialized memory objects, shielding

pending entries from accidental interface re-renders until the transaction is explicitly saved.

- **Identity Transmutation Module:** Exposes interactive controls that allow users to update account details without invalidating the underlying system session. It also handles data filtering to show the active user's inventory, purchases, and bookmarked assets.
- **Asynchronous Negotiation Engine:** Orchestrates chat threads using automated English conversation patterns. It runs background delays alongside a semantic keyword analysis engine to simulate realistic live-agent response delays.

3. Operational Protocols: How to Run the System

Deploying and maintaining this software package locally or in production environments requires the installation of the Flutter Software Development Kit (SDK) and the Dart compilation environment.

3.1 Local Environment Configuration & Dependency Ingestion

Before launching the application execution pipeline, the local engineering workspace must parse the configuration blueprint to fetch required core packages. Open a terminal path at the project's root folder and execute the dependency initialization directive:

flutter pub get

This procedure connects to the package distribution server, maps all dependencies, and compiles the native resource configurations required for multiplatform layout generation.

3.2 Executing the Application in Target Debug Mode

To run the platform inside a local sandbox with hot-reload monitoring activated, input the execution command:

flutter run

For web-centric target optimization, append an explicit platform identifier flag to initialize the runtime within a secure local browser environment:

flutter run -d chrome

This pipeline spins up an active compilation service that pushes interface updates to the target device instantly when code updates are saved, speeding up iterative debugging.

3.3 Constructing and Compiling Production Build Assets

When compilation assets are ready for public hosting or cloud deployment, developers must run the production build utility:

flutter build web --web-renderer html --release

The incorporation of the --web-renderer html flag optimizes the output size by prioritizing native HTML5 elements, CSS rules, and canvas abstractions. This approach reduces initial script download overhead, maximizes loading speeds, and improves search engine crawlability compared to heavier graphic-intensive engines.

The pipeline produces minimized, deployment-ready production bundles located within the project build hierarchy under the production path.

4. Library Inventory & Dependency Topology

To maintain strict performance controls, avoid package bloating, and guarantee high-speed operation on restricted mobile or web clients, the app relies almost entirely on foundational system packages.

4.1 Foundational UI and Core Architecture Packages

- **Material Design Framework Componentry:** Supplies the semantic UI design system tokens, typography scales, accessibility guidelines, and reactive UI elements used throughout the app. It standardizes cross-platform visual presentations without requiring custom CSS definitions.
- **Foundational Diagnostic Utilities:** Provides foundational debugging utilities, multi-device layout constraints, and underlying communication protocols that bridge the declarative codebase with target browser engines.

4.2 Asynchronous Tasks and Threading APIs

- **Asynchronous Contract Implementation APIs:** Supplies the underlying threading architectures, asynchronous microtask hooks, and delayed schedule operators required to simulate system background processes. This package powers the automated chat bot, letting responses process asynchronously without locking or interrupting the main layout thread.

5. Justification of Technical Choices & Architectural Patterns

Every design decision in this iteration is chosen to resolve specific engineering trade-offs, prioritizing UI responsiveness, memory safety, and future backend compatibility.

5.1 Centralized In-Memory Global Datastore Architecture

Utilizing a centralized, static memory class to broker application state instead of immediate cloud server connections serves an intentional engineering purpose: Eliminating structural network delays from frontend testing.

- Forcing early prototypes to wait on HTTP network request-response lifecycles can obscure UI rendering regressions and performance bottlenecks.
- Operating an in-memory data store provides a fast, predictable environment where data mutation rules are completely synchronous.
- Because the data collection layouts mirror standard relational tables, developers can swap this in-memory layer for remote database queries or a structured state manager later with minimal modifications to the UI **layer.**

5.2 Transitioning to Explicit State Retention Management (Stateful Paradigm)

Converting critical application modules from stateless presentation boxes into active state engines handles data volatility concerns when capturing user inputs:

- In a purely declarative environment, stateless components risk wiping typed data buffers if an unrelated layout shift forces an un-cached redrawing of the view tree.
- Moving to stateful components pairs data layers with dedicated memory controllers. This ensures user entries are safely cached during interface changes and only saved once form validation rules are met.

5.3 Asynchronous Microtask Delayed Chat Threading for User Engagement

Integrating delayed microtask execution pipelines within the client communication view solves an immersion issue common in static mockups:

- Instantaneous automated responses ruin user immersion by making it obvious that chat partners are hardcoded text strings.
- By introducing an artificial thread processing delay, the system mimics real human typing intervals.
- Offloading this lookup logic to an asynchronous future prevents computational stalls on the main UI thread. This approach ensures text input layouts remain fully reactive and smooth while the bot computes its semantic response.

5.4 Choice of Web Renderer Profile (HTML Profile vs. CanvasKit)

Opting for the HTML web rendering profile over CanvasKit for production builds balances web payload optimization with layout accuracy:

- While CanvasKit offers pixel-perfect drawing by compiling heavy graphics code into WebAssembly, it incurs a significant initial load time penalty.
- The HTML renderer profile leverages native browser optimization hooks, lowering initial data requirements and ensuring fast page loads on variable network connections. This approach fits standard marketplace platforms that prioritize text rendering, search engine visibility, and fast loading speeds over complex graphic simulations.